

Fig. 12: sim.do command window

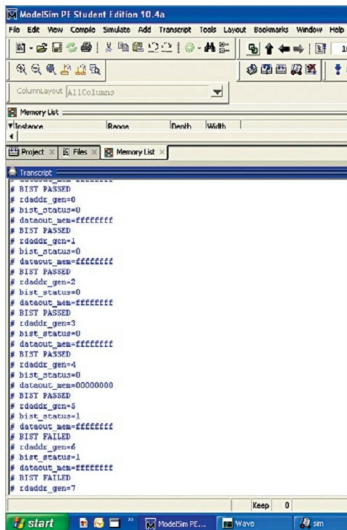


Fig. 13: Transcript window (result shown)

in this result window (Fig. 13).

To observe stuck-at fault 0, we intentionally forced a memory location to invert input data in the example (refer tb.v for the same). Once a fault is detected, BIST will indicate as failed in the result even if there is no fault in the remaining memory locations.

Thus, the result of BIST design is verified from the transcript window as shown in Fig. 13.

This project on BIST for stuck-at-fault has been verified by Nithin Kumar Guggilla, manager at Xilinx India. He says, "Since FPGAs can be programmed with any design on the fly, if needed, designers can implement their own BIST using pseudo-random or pre-calculated generators and signature in their RTL. This would be implemented in look-up tables and flip-flops, which will create some overhead in addition to the actual resources needed for the design."

Xilinx FPGAs also provide dedicated memory that can be used in various applications. There is a built-in error-correction capability for Xilinx block RAMs. Each block RAM has an error-correcting code circuitry associated with it. This can be enabled to correct single-bit

done for a single bit.

Among the various modes (single port, simple dual port and true dual port) supported by block RAM, error-correcting code is supported in simple dual-port mode. In this mode, the block RAM can be configured as 512 x 64 RAM. The extra eight bits in the 72-bit wide RAM are used towards built-in Hamming code-error correction.

Sample code. If you want to use error-correcting code facility of Xilinx BRAM, set the corresponding parameters/ports during instantiation, as shown in the following Verilog example:

```
ramb36e2 {  
...  
.en_ecc_pipe("true"), // enabled  
pipeline on ecc, for higher  
performance  
.en_ecc_read("true"), // enabled ecc  
for read  
.en_ecc_write("true"), // enabled ecc  
for write  
...  
.read_width_a(72), // read width port  
a - 72 in sdp mode  
.write_width_a(0),  
.write_width_b(72) // write width  
port b - 72 in sdp mode  
mem_reg  
{  
// other ports  
.....  
.eccpipece(1'b1), // to enable ecc  
pipelines, connect enable or 1  
.casoutbitter(), // use in  
cascade mode  
.casoutdbitter(), // use in  
cascade mode  
.sbitter(sbitter), // single bit  
error status  
.dbitter(dbitter), // double bit  
error status  
.eccparity(eccparity) // parity bits  
};
```

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download at source.efymag.com



Nidhi Kathuria is a senior application engineer at EFY Tech Center, New Delhi